Estudo Dirigido — openEuler, do básico ao avançado

Oct 13, 2026 · @Michel Dias

1. Visão geral

Em dez módulos e um desafio final, você vai do primeiro login até a operação de um servidor openEuler endurecido, com serviço próprio, firewall, SELinux, backups agendados e ajuste de desempenho, sempre sobre o mesmo laboratório.

O openEuler é a distribuição Linux de código aberto iniciada pela Huawei e hoje mantida pela OpenAtom Foundation, com foco em servidores, nuvem e borda. Na Huawei ICT Competition, a trilha Computing cobra openEuler e openGauss em partes iguais. Os pontos de conhecimento de openEuler vão da operação básica (shell, usuários, pacotes, armazenamento, rede e serviços) até segurança (SELinux, endurecimento, secGear), virtualização e contêineres (StratoVirt, iSulad), monitoramento e ajuste de desempenho (A-Tune) e atualização do kernel sem reinicialização (exame da trilha Computing, edição 2022–2023). Este estudo segue essa lista: os módulos 1 a 8 cobrem a operação, que vale para qualquer Linux da família RPM, e os módulos 9 e 10 cobrem o que é próprio do openEuler.

Ao final, você deve ser capaz de:

- Instalar o openEuler em uma máquina virtual e se orientar no sistema.
- Manipular arquivos e extrair informação de logs e dados com `find`, `grep`, `sed` e `awk`.
- Administrar usuários, grupos, permissões, ACLs e `sudo`.
- Criar e operar serviços com `systemd` e ler logs com `journalctl`.
- Instalar e auditar pacotes com `dnf` e `rpm`, e configurar repositórios.
- Configurar rede, SSH e `firewalld`.
- Particionar discos, criar sistemas de arquivos e gerenciar volumes com LVM.
- Automatizar tarefas com shell script, `cron` e temporizadores do `systemd`.
- Operar o SELinux, aplicar o endurecimento do openEuler e auditar o sistema.
- Usar A-Tune, iSulad e os demais recursos próprios do openEuler.

Como o estudo funciona. Cada tarefa descreve o resultado esperado e aponta os comandos a pesquisar, sem trazer a solução. Cada módulo termina com um ponto de conferência: números ou estados que o seu sistema precisa reproduzir. Se não bater, o erro está naquele módulo.

2. Carga horária, regras e entrega

O estudo soma cerca de 24 horas de trabalho efetivo. O prazo sugerido é de dez dias corridos, a confirmar pelo tutor.

Módulo	Tema	Nível	Tempo	Peso
1	Primeiros passos no sistema	Básico	1 h 30	5%
2	Arquivos, diretórios e texto	Básico	2 h 30	10%
3	Usuários, grupos e permissões	Básico	2 h	10%
4	Processos, serviços e logs	Intermediário	2 h	10%
5	Pacotes e repositórios	Intermediário	1 h 30	7%
6	Rede, SSH e firewall	Intermediário	2 h	10%
7	Armazenamento e LVM	Intermediário	2 h	10%
8	Shell script e automação	Intermediário	2 h 30	10%
9	Segurança	Avançado	2 h 30	10%
10	Desempenho e recursos do openEuler	Avançado	2 h 30	8%
Final	Desafio integrador	Avançado	3 h	10%

Regras

- Atividade individual. Discutir com colegas é permitido; copiar comandos e scripts, não.
- Consulta livre à documentação do openEuler, às páginas `man` e a qualquer material de estudo.
- Faça os módulos na ordem: cada um usa o que o anterior deixou no sistema.
- Toda tarefa é respondida com o comando executado e a saída obtida, copiados do terminal. Saída digitada à mão não vale.
- Trabalhe com um usuário comum e use `sudo` quando precisar de privilégio. Use `root` direto só onde o roteiro pedir.
- Tire um instantâneo (snapshot) da máquina virtual depois de preparar o laboratório e antes dos módulos 7 e 9. Se quebrar o sistema, volte ao instantâneo e refaça o módulo.

O que entregar

1. `relatorio.md`: para cada tarefa, o código (`T3.4`), os comandos e a saída relevante, além das respostas das perguntas para pensar e da tabela de pontos de conferência preenchida.

2. Os scripts criados nos módulos 8 e 10 e no desafio, e as unidades `systemd` dos módulos 4 e 8.
3. `evidencias/`: a saída de `verificar.sh`, o script de autoverificação do desafio final.

Para capturar saídas longas, use `comando | tee -a ~/evidencias/mXX.txt` ou o `script` do pacote `util-linux`.

3. Ambiente

Você precisa de uma máquina virtual com openEuler 24.03 LTS SP4, a versão LTS mais recente, e do pacote `lab_nuvemif.tar.gz`, fornecido pelo tutor. Um contêiner não basta: os módulos de serviços, rede, armazenamento e segurança precisam de `systemd`, de um disco extra e de um kernel próprio.

Passo 1 — Baixar a imagem. Na página de downloads do openEuler, escolha openEuler 24.03 LTS SP4, arquitetura x86_64 (ou AArch64, em computadores ARM), imagem *Offline Standard ISO*.

Passo 2 — Criar a máquina virtual. VirtualBox, VMware Workstation, KVM ou outro hipervisor, com:

- 2 vCPUs, 4 GB de memória e um disco de 40 GB.
- Um segundo disco de 2 GB, vazio, que só será usado no módulo 7.
- Rede em modo bridge ou NAT com redirecionamento da porta 22, para acesso por SSH.

Passo 3 — Instalar. Siga o instalador (Anaconda): idioma, fuso horário, disco de 40 GB com particionamento automático, seleção de software *Server* (sem interface gráfica), senha de `root` e um usuário comum com o seu nome, marcado como administrador. Ao terminar, reinicie e entre com o seu usuário.

Passo 4 — Preparar o laboratório. Copie o pacote para a máquina (por `scp`, por pasta compartilhada ou baixando-o de dentro da VM) e execute:

```
tar xzf lab_nuvemif.tar.gz
cd lab_nuvemif
sudo bash preparar_lab.sh
```

O script cria `/opt/nuvemif` e termina mostrando `3392 linhas em dados/uso_nuvem.csv`, `6000 linhas em logs/coletor.log` e `60 arquivos em arquivos/`. Ele pode ser executado de novo a qualquer momento para refazer o laboratório do zero; só não desfaz o que você criou fora de `/opt/nuvemif`.

Passo 5 — Instantâneo. Tire um instantâneo da máquina agora, chamado `lab-pronto`.

Outros ambientes. O estudo foi escrito para a 24.03 LTS SP4, mas funciona em qualquer openEuler 22.03 LTS ou posterior; mudam versões de pacotes e de kernel. Para praticar apenas os módulos 2, 5 e 8 em qualquer computador com Docker, a imagem oficial serve: `docker run -it --name oe openeuler/openeuler:24.03-lts-sp4 bash`, seguido do mesmo `preparar_lab.sh`.

Documentação: guia de instalação em docs.openeuler.org e a imagem openeuler/openeuler no Docker Hub.

4. O laboratório NuvemIF

A NuvemIF é uma nuvem fictícia que hospeda os projetos de um instituto. O laboratório é o servidor de coleta dela: um agente registra o uso diário das instâncias e grava logs. Tudo fica em `/opt/nuvemif`, criado pelo script de preparação. Os dados são sintéticos.

Caminho	Conteúdo	Usado nos módulos
`dados/uso_nuvem.csv`	Uso diário de 135 instâncias em setembro de 2026: 3391 registros e 14 colunas, separados por vírgula	2, 8, desafio
`logs/coletor.log`	6000 linhas do agente coletor, com níveis INFO, WARN e ERROR	2, 8, 9
`arquivos/`	60 relatórios de tamanhos, extensões e datas variados; três deles com permissão errada	2, 3
`bin/coletor.sh`	Script que simula o agente: grava um batimento a cada 10 segundos	4, 9, desafio
`scripts/relatorio_quebrado.sh`	Script com erros plantados	8
`config/coletor.conf`	Configuração do agente	2, 8
`config/segredos.env`	Credenciais de teste, gravadas com permissão errada de propósito	3, 9
`docs/README.txt`	Descrição do laboratório	2

Colunas de `uso_nuvem.csv`, na ordem: `id_registro`, `data`, `instancia_id`, `projeto_id`, `regiao`, `flavor`, `so`, `cpu_pct`, `mem_uso_gb`, `horas_ativas`, `custo_brl`, `status`, `ambiente`, `equipe`. A primeira linha é o cabeçalho.

Formato de `coletor.log`, uma linha por evento:

```
2026-09-03 14:22:07 [WARN] coletor[4000]: instancia=ecs-0755edc0 disco acima de 90% em /var
```

Os campos são data, hora, nível entre colchetes, processo com PID, instância e mensagem. Mensagens de erro incluem `timeout ao conectar em IP`, `falha de autenticacao usuario=X origem=IP` e `reinicio do agente apos falha`.

Quem fez o estudo dirigido de SQL e openGauss vai reconhecer os dados: são os mesmos da tabela `uso_diario`, e os totais batem.

5. Módulo 1 — Primeiros passos no sistema (1 h 30)

Nível básico. Você aprende a se orientar: que sistema é este, que recursos tem, como pedir ajuda e como editar um arquivo.

T1.1 — Identificar o sistema. Descubra o nome e a versão da distribuição, a versão do kernel, a arquitetura e o nome da máquina. Compare o conteúdo de `/etc/os-release` com a saída de `hostnamectl`.

Comandos: `cat /etc/os-release`, `uname -r`, `uname -m`, `hostnamectl`.

T1.2 — Recursos. Quantos processadores, quanta memória, que discos e partições existem, quanto espaço livre há em `/`, há quanto tempo o sistema está ligado?

Comandos: `lscpu`, `free -h`, `lsblk`, `df -h`, `uptime`.

T1.3 — Quem sou eu. Mostre o seu usuário, os grupos a que pertence, o shell em uso e o diretório pessoal. Use `sudo -i` para abrir um shell de `root`, observe como o prompt muda e saia.

Comandos: `id`, `whoami`, `echo $SHELL`, `echo $HOME`, `sudo -i`, `exit`.

T1.4 — Pedir ajuda. Leia a página de manual do `ls` e descubra a opção que ordena por tamanho. Descubra o que faz a seção 5 do manual e leia `man 5 passwd`. Use `--help` e `apropos` para achar o comando que mostra o calendário.

Comandos: `man`, `man -k` ou `apropos`, `--help`, `type`, `which`.

T1.5 — Data e fuso. Ajuste o fuso horário para `America/Sao_Paulo` e confira se o relógio está sincronizado por NTP.

Comandos: `timedatectl`, `timedatectl set-timezone`, `date`.

T1.6 — Editor. Com o `vim`, crie `~/notas.txt` com três linhas: seu nome, a versão do kernel e o fuso horário. Pratique entrar e sair do modo de inserção, salvar, sair sem salvar, buscar e substituir.

Comandos: `vim`, teclas `i`, `Esc`, `:w`, `:q`, `:q!`, `/texto`, `:%s/a/b/g`.

T1.7 — Histórico e atalhos. Liste o histórico, execute de novo um comando pelo número, busque no histórico com Ctrl+R, complete nomes com Tab, limpe a tela. Crie um apelido `ll` para `ls -la` e torne-o permanente.

Comandos: `history`, `!n`, `alias`, `~/.bashrc`.

Perguntas para pensar

1. Qual é a diferença entre `su -`, `sudo -i` e `sudo comando`? Por que o roteiro pede para trabalhar com usuário comum?
2. O openEuler 24.03 LTS usa o kernel 6.6. O que significa LTS para a distribuição e para o kernel, e por quanto tempo cada um recebe atualizações?

Ponto de conferência 1

Tarefa	Verificação	Valor esperado
T1.1	`ID` e `VERSION_ID` em `/etc/os-release`	`openEuler`; `24.03`
T1.1	Início de `uname -r`	`6.6.`
T1.2	Discos em `lsblk`	Dois: o do sistema e o de 2 GB, sem partições
T1.5	`Time zone` em `timedatectl`	`America/Sao_Paulo`
T1.6	`wc -l ~/notas.txt`	3

6. Módulo 2 — Arquivos, diretórios e texto (2 h 30)

Nível básico. Trabalhe em `/opt/nuvemif`. A maior parte do trabalho de um administrador é achar o arquivo certo e extrair dele a informação certa.

T2.1 — Navegar e copiar. Explore a árvore do laboratório com `ls -la`, `tree` (instale se faltar) e `du -sh`. Copie `arquivos/` para `~/trabalho/arquivos`, preservando datas e permissões, e trabalhe na cópia nas tarefas T2.2 e T2.3.

Comandos: `cd`, `ls`, `cp -a`, `mkdir -p`, `mv`, `rm -r`, `du`.

T2.2 — Encontrar por critério. Em `~/trabalho/arquivos`, quantos arquivos têm mais de 100 KB? Quantos terminam em `.tmp`? Quantos foram modificados antes de 01/09/2026? Quantos podem ser escritos por qualquer usuário? Quantos `.bak` têm mais de 1 MB? Depois apague os `.tmp` com um único comando.

Comandos: `find` com `-size`, `-name`, `-newermt`, `-perm`, `-type`, `-delete` ou `-exec`.

T2.3 — Compactar. Crie `~/trabalho/arquivos.tar.gz` com o conteúdo de `arquivos/`, liste o conteúdo sem extrair e extraia um único arquivo em `/tmp`. Compare o tamanho do pacote com `gzip` e com `xz`.

Comandos: `tar czf`, `tar tzf`, `tar xzf ... caminho`, `tar cJf`.

T2.4 — Contar no log. Em `logs/coletor.log`: quantas linhas há no total e por nível (INFO, WARN, ERROR)? Quantas contêm `timeout ao conectar`? Quantos eventos ocorreram em 16/09/2026?

Comandos: `wc -l`, `grep -c`, `grep -E`, `awk '{print $3}' | sort | uniq -c`.

T2.5 — Encontrar o atacante. Entre as linhas de `falha de autenticacao`, qual endereço de origem aparece mais vezes, e quantas? Qual instância tem mais linhas de erro?

Comandos: `grep -o`, `sort`, `uniq -c`, `sort -rn`, `head`.

T2.6 — Somar no CSV. Em `dados/uso_nuvem.csv`: o custo total do mês, o custo do projeto PRJ-004, o custo do dia 16/09/2026, o número de registros com status `error` e o número de instâncias distintas. Lembre que a primeira linha é cabeçalho.

Comandos: `awk -F,` com `NR > 1`, `$11`, somas em variáveis, `END`, `printf`; `cut -d,`, `sort -u`.

T2.7 — Agrupar. Com um único `awk`, calcule o custo por projeto e mostre os três maiores. Conte os registros por região com `cut`, `sort` e `uniq`.

T2.8 — Substituir. Em uma cópia do CSV, troque todas as ocorrências de `sa-brazil-1` por `br-south-1` e confira quantas linhas mudaram. Em uma cópia de `config/coletor.conf`, mude `nivel_log=INFO` para `nivel_log=DEBUG` editando no próprio arquivo.

Comandos: `sed 's/a/b/g'`, `sed -i`, `diff`, `grep -c`.

T2.9 — Redirecionar. Grave em `~/trabalho/erros.txt` só as linhas de erro do log. Execute `ls /raiz-inexistente /opt` de modo que a saída normal vá para um arquivo e a mensagem de erro vá para outro. Acrescente uma linha a um arquivo sem apagar o conteúdo. Explique `2>&1` e `/dev/null`.

Comandos: `>`, `>>`, `2>`, `2>&1`, `|`, `tee`.

T2.10 — Ligações. Crie uma ligação simbólica `~/uso.csv` para o CSV e uma ligação física para `docs/README.txt`. Compare `ls -li` dos dois casos, apague os originais em cópias de teste e veja o que acontece com cada ligação.

Comandos: `ln -s`, `ln`, `ls -li`, `readlink`, `stat`.

Perguntas para pensar

1. Por que `grep PRJ-004 uso_nuvem.csv` e `awk -F, '$4 == "PRJ-004"'` podem dar resultados diferentes em um CSV qualquer? Aqui eles coincidem; em que situação não coincidiriam?
2. `rm -rf` não pergunta nada. Que hábitos reduzem o risco de apagar o diretório errado?

Ponto de conferência 2

Tarefa	Verificação	Valor esperado
T2.1	`du -sh arquivos`	18M
T2.2	Mais de 100 KB; `.tmp`; antes de 01/09; graváveis por todos; `.bak` acima de 1 MB	28; 12; 38; 3; 5
T2.2	Arquivos restantes após apagar os `.tmp`	48
T2.4	Total; INFO; WARN; ERROR	6000; 4790; 616; 594
T2.4	`timeout ao conectar`; eventos em 16/09	301; 200
T2.5	Origem mais frequente e contagem	198.51.100.23; 129
T2.5	Linhas de erro da instância mais frequente	10 (há empate)
T2.6	Custo total; PRJ-004; dia 16/09	65984.13; 12486.39; 2865.97
T2.6	Registros `error`; instâncias distintas	90; 135
T2.7	Registros em `sa-brazil-1`	1718
T2.8	Linhas alteradas pelo `sed`	1718

7. Módulo 3 — Usuários, grupos e permissões (2 h)

Nível básico. A equipe da NuvemIF vai usar o servidor, e o agente coletor precisa de uma identidade própria. Você decide quem acessa o quê.

T3.1 — Contas. Crie o grupo `nuvemif` com GID 2500 e os usuários `ana` (UID 2501), `bruno` (UID 2502) e `carla` (UID 2503), todos com o grupo `nuvemif` como grupo suplementar e `bash` como shell, com senha definida. Crie também a conta de sistema `coletor`, sem senha, sem diretório pessoal e sem shell de login, para o serviço do módulo 4. Confira em `/etc/passwd`, `/etc/group` e `/etc/shadow` o que cada comando gravou.

Comandos: `groupadd -g`, `useradd -u -G -s`, `useradd -r -s /sbin/nologin`, `passwd`, `id`, `getent`.

T3.2 — Política de senhas. Faça a senha de `ana` expirar a cada 90 dias, com aviso de 7 dias. Bloqueie a conta de `bruno` e confira que ele não consegue entrar; depois desbloqueie. Defina uma data de expiração da conta de `carla` para 31/12/2026. Identifique em `/etc/login.defs` o tamanho mínimo de senha e a validade padrão.

Comandos: `chage -M -W -E -l`, `usermod -L`, `usermod -U`, `passwd -S`.

T3.3 — Corrigir o que está errado. O arquivo `config/segredos.env` pode ser lido por qualquer usuário e três relatórios em `arquivos/` podem ser alterados por qualquer um. Deixe `segredos.env` com dono `coletor`, grupo `nuvemif` e leitura apenas para dono e grupo. Encontre e corrija os relatórios com um único `find`.

Comandos: `chown`, `chgrp`, `chmod` em octal e em símbolos, `find -perm -o+w -exec chmod o-w {} +`.

T3.4 — Diretório compartilhado. Crie `/opt/nuvemif/compartilhado`, do grupo `nuvemif`, em que todo arquivo novo nasça com o grupo `nuvemif`, só membros do grupo entrem, e ninguém apague arquivo dos outros. Prove com `ana` criando um arquivo e `bruno` tentando apagá-lo.

Comandos: bit SGID e bit sticky, `chmod 3770` ou `chmod g+s,o+t`, `su - usuario`.

T3.5 — Máscara. Qual é a `umask` do seu usuário e a de `root`? Crie um arquivo e um diretório e explique as permissões resultantes a partir da máscara. Mude a `umask` da sessão para `077` e repita.

T3.6 — ACL. `carla` não é membro de nenhum grupo com acesso a `dados/`, mas precisa ler os CSVs. Conceda leitura a ela sem mudar dono, grupo nem permissões do diretório. Mostre a ACL resultante e o sinal que o `ls -l` passa a exibir.

Comandos: `setfacl -m u:carla:rx`, `setfacl -R`, `getfacl`.

T3.7 — Privilégio delegado. Permita que `ana` execute `systemctl restart coletor` e `journalctl -u coletor` sem senha, e que o grupo `nuvemif` execute `dnf` com senha. Use um arquivo próprio em `/etc/sudoers.d/`, validado pelo editor adequado. Teste com `sudo -l -U ana`.

Comandos: `visudo -f /etc/sudoers.d/nuvemif`, `sudo -l`.

T3.8 — Bits especiais. Liste os executáveis com SUID em `/usr/bin` e explique por que `passwd` precisa dele. Mostre as permissões de `/tmp` e explique o `t` no fim.

Comandos: `find /usr/bin -perm -4000`, `ls -ld /tmp`.

Perguntas para pensar

1. Por que o serviço roda como `coletor` e não como `root`? O que um invasor ganharia em cada caso?

2. ACL, grupo suplementar e `sudo` resolvem problemas diferentes. Dê um exemplo de cada um no cenário da NuvemIF.

Ponto de conferência 3

Tarefa	Verificação	Valor esperado
T3.1	`id ana`	`uid=2501(ana)` e `2500(nuvemif)` entre os grupos
T3.1	Shell de `coletor` em `getent passwd coletor`	`/sbin/nologin`
T3.2	`Maximum number of days` em `chage -l ana`	90
T3.3	`ls -l config/segredos.env`	`-rw-r----- coletor nuvemif`
T3.3	`find /opt/nuvemif -type f -perm -o+w`	Nenhum arquivo
T3.4	`ls -ld compartilhado`	`drwxrws--T` ou `drwxrws--t`, grupo `nuvemif`
T3.6	`getfacl dados` contém	`user:carla:r-x`
T3.7	`sudo -l -U ana` lista	Os dois comandos, com `NOPASSWD`

8. Módulo 4 — Processos, serviços e logs com systemd (2 h)

Nível intermediário. O agente coletor deixa de ser um script solto e vira um serviço do sistema, que inicia com a máquina, reinicia sozinho e registra o que faz.

T4.1 — Processos. Quantos processos existem agora? Qual é o PID do `sshd` principal e qual é o PID 1? Mostre a árvore de processos e os cinco que mais usam memória.

Comandos: `ps aux`, `ps -ef --forest`, `pgrep`, `pstree`, `top`, `ps -eo pid,comm,%mem --sort=-%mem`.

T4.2 — Em primeiro e segundo plano. Execute `COLETOR_LOG=/tmp/coletor-teste.log bin/coletor.sh &` (a variável manda o log para um arquivo em que o seu usuário pode escrever), liste os trabalhos, traga-o para o primeiro plano, suspenda com Ctrl+Z, retome em segundo plano. Encerre-o com `kill -TERM` e veja no log o que o script escreveu ao receber o sinal. Execute de novo e encerre com `kill -KILL`: o que mudou? Explique `nohup` e `nice`.

Comandos: `&`, `jobs`, `fg`, `bg`, `kill -TERM`, `kill -KILL`, `nohup`, `nice`, `renice`.

T4.3 — Unidade de serviço. Crie `/etc/systemd/system/coletor.service` para executar `/opt/nuvemif/bin/coletor.sh` como usuário `coletor`, depois da rede, reiniciando em caso de falha, habilitado no alvo multiusuário. Habilite e inicie. A primeira tentativa falha: descubra o motivo no journal, corrija e tente de novo.

Comandos: seções `[Unit]`, `[Service]` e `[Install]`, `User=`, `ExecStart=`, `Restart=on-failure`, `systemctl daemon-reload`, `systemctl enable --now`, `systemctl status`.

T4.4 — Ler o journal. Mostre as mensagens do serviço, acompanhe-as em tempo real, filtre as da última hora e as de prioridade erro de todo o sistema. Torne o journal persistente entre reinicializações e confira o tamanho que ele ocupa.

Comandos: `journalctl -u`, `-f`, `--since`, `-p err`, `-b`, `--disk-usage`, `/etc/systemd/journald.conf`.

T4.5 — Resiliência. Com o serviço ativo, mate o processo com `kill -9`. Observe o que o `systemd` faz nos segundos seguintes e quantas vezes o serviço já foi iniciado. Reinicie a máquina e confira que o serviço subiu sozinho.

T4.6 — O sistema como um todo. Qual é o alvo padrão de inicialização? Quantos serviços estão em execução? Quais os cinco que mais demoraram para iniciar? Mascare um serviço que não será usado e mostre a diferença entre desabilitar e mascarar.

Comandos: `systemctl get-default`, `list-units --type=service --state=running`, `systemd-analyze blame`, `systemctl mask`, `systemctl list-dependencies`.

T4.7 — Rotação de logs. O arquivo `/var/log/nuvemif/coletor-servico.log` cresce para sempre. Configure o `logrotate` para rotacioná-lo diariamente, guardando sete cópias compactadas, sem interromper o serviço. Force uma rotação para testar.

Comandos: `/etc/logrotate.d/nuvemif`, `copytruncate`, `logrotate -f`.

Perguntas para pensar

1. Qual é a diferença entre `SIGTERM` e `SIGKILL`, e por que um serviço bem escrito trata o primeiro?
2. O que `Restart=on-failure` cobre e o que não cobre? Quando `Restart=always` seria melhor, e quando seria pior?

Ponto de conferência 4

Tarefa	Verificação	Valor esperado
T4.1	Nome do PID 1	`systemd`
T4.2	Última linha do log após `kill -TERM`	`parando por SIGTERM`

Tarefa	Verificação	Valor esperado
T4.3	`systemctl is-active coletor`; `is-enabled`	`active`; `enabled`
T4.3	Linha `coletor iniciado` no log mostra	`usuario=coletor`
T4.5	Estado 15 segundos depois do `kill -9`	`active`, com novo PID
T4.6	`systemctl get-default`	`multi-user.target`
T4.7	Arquivo rotacionado em `/var/log/nuvemif/`	Existe, com sufixo de data ou `.1`, compactado

9. Módulo 5 — Pacotes e repositórios com dnf e rpm (1 h 30)

Nível intermediário. O openEuler usa pacotes RPM e o gerenciador `dnf`, como as distribuições da família Red Hat, com repositórios próprios em `repo.openeuler.org`.

T5.1 — Repositórios. Liste os repositórios habilitados e abra o arquivo que os define em `/etc/yum.repos.d/`. Identifique o repositório principal (OS), o de atualizações (update) e o EPOL, e o que significam `baseurl`, `gpgcheck` e `gpgkey`. Atualize o cache de metadados.

Comandos: `dnf repolist`, `dnf repolist -v`, `dnf makecache`, `dnf config-manager`.

T5.2 — Procurar e instalar. Procure o pacote que fornece o comando `tree` e o que fornece `nginx`. Veja as informações e as dependências do `nginx` antes de instalar. Instale os dois. Quantos pacotes a instalação do `nginx` trouxe junto?

Comandos: `dnf search`, `dnf provides`, `dnf info`, `dnf repoquery --requires`, `dnf install`.

T5.3 — Auditar o que está instalado. Quantos pacotes estão instalados? Que pacote fornece `/usr/bin/awk`? Liste os arquivos do pacote `nginx` e os de configuração. Verifique se algum arquivo do pacote `bash` foi alterado desde a instalação. Mostre versão e data de instalação do kernel.

Comandos: `rpm -qa`, `rpm -qf`, `rpm -ql`, `rpm -qc`, `rpm -V`, `rpm -qi`.

T5.4 — Histórico e retorno. Veja o histórico de transações do `dnf`, os detalhes da transação que instalou o `nginx` e desfaça-a. Confira que o `nginx` saiu. Instale de novo, porque o módulo 6 precisa dele.

Comandos: `dnf history`, `dnf history info N`, `dnf history undo N`.

T5.5 — Atualizações. Liste as atualizações disponíveis e as de segurança. Aplique apenas as de segurança. Explique o que `dnf update` faria com o kernel e como escolher o kernel no boot depois.

Comandos: `dnf check-update`, `dnf updateinfo`, `dnf update --security`, `grubby --default-kernel`.

T5.6 — Pacote avulso e grupos. Baixe o RPM do `htop` sem instalar, inspecione-o com `rpm -qpl` e instale a partir do arquivo. Liste os grupos de pacotes disponíveis e o conteúdo de um deles.

Comandos: `dnf download`, `rpm -qpi`, `rpm -qpl`, `dnf install ./arquivo.rpm`, `dnf group list`, `dnf group info`.

T5.7 — Repositório local. Crie `/srv/repo` com os RPMs baixados, gere os metadados e cadastre-o como repositório `local`, sem verificação de assinatura. Instale um pacote a partir dele com os repositórios remotos desabilitados.

Comandos: `createrepo_c`, arquivo `.repo` com `baseurl=file:///srv/repo`, `dnf --disablerepo='*' --enablerepo=local`.

Perguntas para pensar

1. Qual é a divisão de trabalho entre `rpm` e `dnf`? O que o `dnf` faz que o `rpm` não faz?
2. Por que `gpgcheck=1` importa, e o que um repositório sem assinatura permite a um atacante no caminho da rede?

Ponto de conferência 5

Tarefa	Verificação	Valor esperado
T5.1	Repositórios habilitados incluem	`OS`, `everything`, `EPOL`, `update`
T5.2	`rpm -q nginx tree`	As duas versões, sem `not installed`
T5.3	`rpm -qf /usr/bin/awk`	`gawk-...`
T5.4	`rpm -q nginx` após o `undo`	`package nginx is not installed`
T5.6	`rpm -q htop`	Instalado
T5.7	`dnf repolist` inclui	`local`

10. Módulo 6 — Rede, SSH e firewall (2 h)

Nível intermediário. O servidor passa a ser alcançado pela rede: um site de status servido pelo `nginx`, acesso remoto por chave e um firewall que só abre o necessário.

T6.1 — Reconhecer a rede. Mostre as interfaces, os endereços IP, a rota padrão e os servidores DNS. Teste a resolução de nomes e a conectividade com `repo.openeuler.org`.

Comandos: `ip addr`, `ip route`, `nmcli device`, `nmcli connection show`, `cat /etc/resolv.conf`, `ping`, `getent hosts`, `dig` ou `nslookup` (pacote `bind-utils`).

T6.2 — Nome e endereço fixo. Mude o nome da máquina para `nuvemif-srv`. Com o `NetworkManager`, configure a interface com endereço IPv4 fixo, gateway e DNS adequados à sua rede virtual, de forma persistente, e reative a conexão. Acrescente em `/etc/hosts` o nome `painel.nuvemif.local` apontando para o endereço da máquina.

Comandos: `hostnamectl set-hostname`, `nmcli connection modify ... ipv4.addresses ipv4.gateway ipv4.dns ipv4.method manual`, `nmcli connection up`.

T6.3 — Portas abertas. Inicie e habilite o `nginx` e descubra em que porta ele escuta e com que processo. Liste todas as portas TCP e UDP em escuta e identifique o processo de cada uma.

Comandos: `systemctl enable --now nginx`, `ss -tulnp`, `curl -I http://localhost`.

T6.4 — Site de status. Faça o `nginx` servir, na porta 8080, o diretório `/srv/painel`, com uma página `index.html` que mostre o nome da máquina e a data de geração. Teste com `curl` na própria máquina. Se a resposta for de permissão negada, registre o erro: o módulo 9 explica.

Comandos: arquivo em `/etc/nginx/conf.d/`, `nginx -t`, `systemctl reload nginx`, `curl http://localhost:8080`.

T6.5 — Firewall. Verifique se o `firewalld` está ativo e qual é a zona padrão. Liste o que ela permite. Libere a porta 8080 de forma permanente, recarregue e teste de outra máquina (o hospedeiro). Depois, remova a liberação da porta e, em vez dela, libere o serviço `http` e mude o `nginx` para a porta 80.

Comandos: `firewall-cmd --state`, `--get-default-zone`, `--list-all`, `--add-port=8080/tcp --permanent`, `--reload`, `--add-service=http`, `--remove-port`.

T6.6 — SSH por chave. No hospedeiro, gere um par de chaves e instale a chave pública para o seu usuário na VM. Entre sem senha. Depois, no servidor, desabilite o login de `root` e a autenticação por senha, recarregue o `sshd` e prove que uma conexão por senha é recusada. Mantenha uma sessão aberta enquanto testa, para não se trancar fora.

Comandos: `ssh-keygen`, `ssh-copy-id`, `/etc/ssh/sshd_config` (`PermitRootLogin`, `PasswordAuthentication`), `sshd -t`, `systemctl reload sshd`.

T6.7 — Transferir e diagnosticar. Copie `/opt/nuvemif/dados/uso_nuvem.csv` para o hospedeiro e um arquivo de volta. Mostre as conexões TCP estabelecidas com a sua sessão SSH. Use `tcpdump` para capturar dez pacotes da porta 80 enquanto faz um `curl`.

Comandos: `scp`, `rsync`, `ss -tnp`, `tcpdump -i any port 80 -c 10`.

Perguntas para pensar

1. Qual é a diferença entre `--permanent` e a configuração em tempo de execução do `firewalld`? O que acontece com uma regra sem `--permanent` depois do `--reload`?
2. Por que liberar o serviço `http` é preferível a liberar a porta 80? E quando a porta é a única opção?

Ponto de conferência 6

Tarefa	Verificação	Valor esperado
T6.2	`hostnamectl --static`	`nuvemif-srv`
T6.2	`nmcli -g ipv4.method connection show <conexão>`	`manual`
T6.3	`ss -tlnp` mostra	`nginx` na porta 80 e `sshd` na 22
T6.5	`firewall-cmd --list-services` inclui	`ssh` e `http`
T6.5	`firewall-cmd --list-ports`	Vazio
T6.6	`sshd -T` contém	`permitrootlogin no` e `passwordauthentication no`
T6.6	`ssh -o PubkeyAuthentication=no usuario@vm`	Recusado (`Permission denied`)

11. Módulo 7 — Armazenamento: discos, sistemas de arquivos e LVM (2 h)

Nível intermediário. O disco de 2 GB que ficou vazio na instalação vai guardar os dados da NuvemIF, com espaço para crescer sem reinstalar nada. Tire um instantâneo antes de começar.

T7.1 — Reconhecer. Identifique o disco vazio (por exemplo, `/dev/sdb` ou `/dev/vdb`), seu tamanho e confirme que não tem partições nem sistema de arquivos. Mostre como o disco do sistema foi particionado na instalação e que sistemas de arquivos usa.

Comandos: `lsblk -f`, `fdisk -l`, `blkid`, `df -hT`, `cat /etc/fstab`.

T7.2 — Particionar. Crie no disco vazio duas partições: uma de 500 MB e outra com o restante. Use tabela GPT.

Comandos: `parted` ou `fdisk`, `partprobe`, `lsblk`.

T7.3 — Sistema de arquivos e montagem. Formate a partição de 500 MB com `ext4` e monte-a em `/mnt/backup`. Torne a montagem permanente pelo UUID, teste o `fstab` sem reiniciar e depois reinicie para confirmar.

Comandos: `mkfs.ext4`, `blkid`, `/etc/fstab`, `mount -a`, `findmnt`, `umount`.

T7.4 — LVM. Com a segunda partição, crie o volume físico, o grupo `vg_nuvemif` e o volume lógico `lv_dados` de 800 MB com `xfs`, montado permanentemente em `/srv/dados`. Mova para lá o conteúdo de `/opt/nuvemif/dados` e deixe uma ligação simbólica no lugar, para que o laboratório continue funcionando.

Comandos: `pvcreate`, `vgcreate`, `lvcreate -L -n`, `mkfs.xfs`, `pvs`, `vgs`, `lvs`, `lvdisplay`.

T7.5 — Crescer sem desmontar. Aumente `lv_dados` em 400 MB e estenda o sistema de arquivos com ele montado. Confira o novo tamanho em `df`. Quanto espaço livre sobrou no grupo?

Comandos: `lvextend -L +400M -r` ou `lvextend` seguido de `xfs_growfs`, `vgs`.

T7.6 — Swap. O servidor tem pouca memória. Crie um arquivo de swap de 512 MB, ative-o e torne-o permanente. Mostre a memória e o swap antes e depois.

Comandos: `fallocate` ou `dd`, `mkswap`, `swapon`, `free -h`, `swapon --show`.

T7.7 — Quem ocupa o espaço. Encontre os dez maiores diretórios abaixo de `/var` e os cinco maiores arquivos do sistema. Mostre o uso de inodes de cada sistema de arquivos e explique quando um disco "cheio" ainda tem bytes livres.

Comandos: `du -xh --max-depth=1 /var | sort -rh`, `find / -xdev -type f -size +50M`, `df -i`.

T7.8 — Quebrar e consertar. Com um instantâneo tirado, coloque uma linha errada no `fstab` (um UUID inexistente) e reinicie. Entre no modo de emergência, corrija o arquivo e volte ao normal. Registre o que viu na tela.

Perguntas para pensar

1. Por que montar pelo UUID, e não por `/dev/sdb1`? O que pode mudar entre uma inicialização e outra?
2. Que vantagens o LVM trouxe na T7.5 em relação a uma partição comum? Que operação o `xfs` não permite e o `ext4` permite?

Ponto de conferência 7

Tarefa	Verificação	Valor esperado
T7.2	`lsblk` do disco de 2 GB	Duas partições, a primeira de 500M
T7.3	`findmnt /mnt/backup` após reiniciar	Montado, tipo `ext4`
T7.4	`lvs`	`lv_dados` em `vg_nuvemif`, 800.00m
T7.4	`ls /opt/nuvemif/dados/uso_nuvem.csv`	Existe, via ligação simbólica
T7.5	`lvs` após a extensão; `df -h /srv/dados`	1.17g; cerca de 1.2G
T7.6	`swapon --show`	Arquivo de 512M ativo
T7.8	Sistema inicializa normalmente após a correção	Sim

12. Módulo 8 — Shell script e automação (2 h 30)

Nível intermediário. O que você fez à mão no módulo 2 vira script, e o script passa a rodar sozinho, na hora certa.

T8.1 — Consertar. `scripts/relatorio_quebrado.sh` deveria mostrar o número de registros e o custo total de um projeto, mas não funciona. Encontre e corrija todos os erros (há pelo menos quatro: um de sintaxe, um de aspas, um de lógica na seleção das linhas e um clássico de variável que perde o valor dentro de um `while` alimentado por `|`). Explique cada um em uma frase.

Comandos: `bash -n`, `bash -x`, `shellcheck` (pacote `ShellCheck`, se disponível).

T8.2 — Escrever com opções. Crie `~/bin/relatorio_custo.sh`, que lê o CSV e aceita `-p PROJETO` (custo e registros do projeto), `-d DATA` (custo e instâncias ativas no dia), `-a ARQUIVO` (outro CSV; o padrão é o do laboratório) e `-h` (ajuda). Sem opções, mostra o custo total. Valide a existência do arquivo e devolva códigos de saída diferentes para uso incorreto e arquivo inexistente. Use `awk` para somar.

Comandos: `getopts`, `case`, `[[ ]]`, `exit`, `$?`, funções.

T8.3 — Alerta de erros. Crie `~/bin/alerta_erros.sh`, que lê `coletor.log`, conta os erros por dia e imprime os dias com mais de 25 erros, no formato `DATA: N erros`. O limite deve poder ser alterado pelo primeiro argumento. Termine com código de saída 1 se houver algum dia acima do limite e 0 caso contrário.

T8.4 — Laços e funções. Crie `~/bin/gera_relatorios.sh`, que cria `/srv/dados/relatorios/` e, para cada projeto presente no CSV, grava `PROJETO.txt` com o número de registros e o custo total, usando uma função para o cálculo. Ao final, imprima quantos arquivos gerou.

Comandos: `for`, `while read`, funções, `$(...)`, `mkdir -p`, `printf`.

T8.5 — Agendar com cron. Agende `alerta_erros.sh` para rodar todos os dias às 07:00, gravando a saída em `~/alertas.log`, no `crontab` do seu usuário. Agende também, em `/etc/cron.d/`, uma limpeza semanal de arquivos `.tmp` em `/opt/nuvemif/arquivos` com mais de 30 dias.

Comandos: `crontab -e`, `crontab -l`, formato de cinco campos, `/etc/cron.d/`, `run-parts`.

T8.6 — Agendar com systemd. Crie `nuvemif-backup.service` (tipo `oneshot`), que compacta `/opt/nuvemif/dados` e `/opt/nuvemif/config` em `/mnt/backup/nuvemif-DATA.tar.gz`, e `nuvemif-backup.timer`, que o dispara diariamente às 02:00 e recupera execuções perdidas. Habilite o temporizador, dispare o serviço manualmente uma vez e confira o arquivo gerado.

Comandos: `Type=oneshot`, `OnCalendar=`, `Persistent=true`, `systemctl enable --now nome.timer`, `systemctl list-timers`, `systemctl start nome.service`.

T8.7 — Scripts que falham bem. Reescreva o início de `gera_relatorios.sh` com `set -euo pipefail` e um `trap` que apague arquivos temporários ao sair. Provoque um erro (CSV inexistente) e mostre que o script para na hora, com mensagem clara.

Perguntas para pensar

1. Quando usar `cron` e quando usar um temporizador do `systemd`? Cite duas coisas que o temporizador faz e o `cron` não.
2. Por que `while read` alimentado por `|` perde variáveis, e quais são as duas formas de evitar isso?

Ponto de conferência 8

Tarefa	Verificação	Valor esperado
T8.1	Script corrigido com `PRJ-004`	`539 registros`, `R$ 12486.39`
T8.2	`relatorio_custo.sh -p PRJ-004`; `-d 2026-09-16`; sem opções	12486.39; 2865.97; 65984.13
T8.2	Código de saída com `-a /nao/existe`	Diferente de 0 e diferente do código de uso incorreto

Tarefa	Verificação	Valor esperado
T8.3	Dias acima de 25 erros; o dia com mais erros	3; 2026-09-12 com 29
T8.4	Arquivos em `/srv/dados/relatorios/`	14
T8.5	`crontab -l`	Linha `0 7 * * *` com o script
T8.6	`systemctl list-timers` inclui; arquivo em `/mnt/backup`	`nuvemif-backup.timer`; `nuvemif-DATA.tar.gz`

13. Módulo 9 — Segurança: SELinux, endurecimento e auditoria (2 h 30)

Nível avançado. O servidor está exposto à rede; agora você limita o que cada processo pode fazer, aplica o endurecimento do openEuler e registra quem mexeu em quê. Tire um instantâneo antes de começar.

T9.1 — Estado do SELinux. Em que modo o SELinux está? Qual é a política carregada? Onde o modo é definido para a próxima inicialização? Alterne para permissivo e de volta para restritivo sem reiniciar, e explique a diferença entre os três modos.

Comandos: `getenforce`, `sestatus`, `setenforce`, `/etc/selinux/config`.

T9.2 — Contexto de arquivos. Volte ao site de status da T6.4, servido de `/srv/painel`. Compare o contexto dos arquivos dele com o de `/usr/share/nginx/html`. Encontre no log de auditoria a negação correspondente e corrija de forma permanente, com uma regra de contexto, e não com `chcon`. Prove com `curl`.

Comandos: `ls -Z`, `ausearch -m AVC -ts recent`, `audit2why`, `semanage fcontext -a -t httpd_sys_content_t`, `restorecon -Rv` (pacote `policycoreutils-python-utils`).

T9.3 — Portas e booleanos. Mude o site de status para a porta 8088 e observe o `nginx` falhar ao recarregar. Registre a porta no SELinux e tente de novo. Liste os booleanos relacionados ao `httpd` e explique o que `httpd_can_network_connect` permite. Deixe o site na porta 80 ao final.

Comandos: `semanage port -l`, `semanage port -a -t http_port_t -p tcp 8088`, `getsebool -a`, `setsebool -P`.

T9.4 — Endurecimento do openEuler. O openEuler traz uma ferramenta de endurecimento com configuração em `/etc/openEuler_security/`. Leia `security.conf` e identifique cinco itens que ele aplica. Acrescente em `usr-security.conf` um item seu

que garanta `PermitRootLogin no` no `sshd_config`, reinicie `openEuler-security.service` e confira o resultado no arquivo e em `/var/log/openEuler-security.log`.

T9.5 — Auditoria. Com o `auditd`, vigie escrita e mudança de atributos em `/opt/nuvemif/config`, de forma persistente, com a chave `nuvemif_config`. Altere `coletor.conf` e encontre o evento, com usuário e comando. Gere um relatório resumido dos eventos de autenticação.

Comandos: `auditctl -w ... -p wa -k`, `/etc/audit/rules.d/`, `augenrules --load`, `ausearch -k`, `aureport -au`.

T9.6 — Senhas e bloqueio. Exija senhas de ao menos 12 caracteres com três classes e bloqueie a conta por 10 minutos após cinco falhas. Teste as duas regras com o usuário `bruno` e desbloqueie-o.

Comandos: `/etc/security/pwquality.conf`, `pam_faillock` em `/etc/pam.d/system-auth` e `password-auth`, `faillock --user bruno`, `faillock --reset`.

T9.7 — Integridade e atualizações. Quais arquivos de configuração de pacotes foram alterados desde a instalação? Há atualizações de segurança pendentes, e com que gravidade? Liste os avisos de segurança (SA) disponíveis para o `openssh`.

Comandos: `rpm -Va | grep '^..5'`, `dnf updateinfo list security`, `dnf updateinfo info`.

T9.8 — secGear. Leia a documentação do secGear e responda em até dez linhas: que problema ele resolve, o que é um ambiente de execução confiável (TEE), que hardware ele exige e em que etapa do desenvolvimento ele entra. Se o laboratório tiver o hardware, instale o pacote e compile o exemplo.

Perguntas para pensar

1. O `nginx` rodava como `root`? Se não, por que as permissões tradicionais não bastaram na T9.2 e o SELinux ainda bloqueou?
2. Por que desligar o SELinux é uma resposta errada a uma negação, e qual é a sequência certa de diagnóstico?

Ponto de conferência 9

Tarefa	Verificação	Valor esperado
T9.1	`getenforce` ao final	`Enforcing`
T9.2	`ls -Z /srv/painel/index.html`	`httpd_sys_content_t`

Tarefa	Verificação	Valor esperado
T9.2	`curl -s -o /dev/null -w '%{http_code}' http://localhost/`	200
T9.3	`semanage port -l` filtrado por `http_port_t`	Inclui 8088
T9.4	`sshd -T` filtrado por `permitrootlogin`	`permitrootlogin no`
T9.5	`auditctl -l`	Regra com `-k nuvemif_config`
T9.6	`faillock --user bruno` após cinco falhas	Cinco registros, conta bloqueada

14. Módulo 10 — Desempenho e recursos próprios do openEuler (2 h 30)

Nível avançado. Aqui entram as ferramentas que fazem do openEuler mais do que "um Linux de servidor": o ajuste automático de desempenho A-Tune, o motor de contêineres iSulad e a virtualização leve StratoVirt. Boa parte das questões de prova sobre openEuler cai aqui.

T10.1 — Medir antes de ajustar. Colete uma linha de base do servidor em repouso: carga, uso de CPU, memória, E/S de disco e rede. Depois gere carga por 60 segundos (`stress-ng` , do repositório, ou um laço de `sha256sum` sobre o CSV) e colete de novo. Que indicador mudou mais?

Comandos: `uptime` , `vmstat 1 5` , `iostat -x 1 5` e `sar` (pacote `sysstat`), `free -h` , `top` , `ss -s` .

T10.2 — Parâmetros do kernel. Consulte `vm.swappiness` , `net.core.somaxconn` e `fs.file-max` . Reduza o `swappiness` para 10 e aumente o `somaxconn` para 4096, de forma persistente, e confira depois de recarregar. Mostre também os limites de arquivos abertos do seu usuário e como elevá-los.

Comandos: `sysctl -a` , `sysctl -w` , `/etc/sysctl.d/90-nuvemif.conf` , `sysctl --system` , `ulimit -n` , `/etc/security/limits.d/` .

T10.3 — A-Tune. Instale `atune` e `atune-engine` , inicie os serviços `atuned` e `atune-engine` e liste os perfis disponíveis. Aplique o perfil `web-nginx-http-long-connection` , veja o que ele mudou (compare `sysctl -a` antes e depois) e desfaça com `rollback` . Execute a análise automática de carga com o site de status sendo acessado em laço e registre o perfil que o A-Tune recomendou.

Comandos: `dnf install atune atune-engine`, `systemctl start atuned atune-engine`, `atune-adm list`, `atune-adm profile`, `atune-adm rollback`, `atune-adm analysis`.

T10.4 — iSulad. Instale e inicie o `iSulad`. Configure em `/etc/isulad/daemon.json` o registro `docker.io` como espelho, baixe a imagem `openeuler/openeuler:24.03-lts-sp4`, execute um contêiner com limite de 256 MB de memória e 1 CPU, entre nele, confira a versão do sistema e saia. Liste contêineres e imagens, pare e remova o contêiner. Compare a sintaxe com a do Docker.

Comandos: `dnf install iSulad`, `systemctl enable --now isulad`, `isula pull`, `isula run -tid --memory 256m --cpus 1 --name teste`, `isula exec -it teste bash`, `isula ps -a`, `isula images`, `isula stop`, `isula rm`.

T10.5 — Contêiner como serviço. Faça o site de status rodar em um contêiner `nginx` gerenciado pelo `iSulad`, com `/srv/painel` montado como volume somente leitura e a porta 8081 publicada. Libere a porta no firewall e teste com `curl`.

Comandos: `isula run -d -p 8081:80 -v /srv/painel:/usr/share/nginx/html:ro`.

T10.6 — Virtualização. Verifique se a máquina virtual expõe virtualização aninhada (`/dev/kvm`). Instale o `stratovirt` e leia a documentação. Responda: o que o StratoVirt tem em comum com o QEMU, em que ele difere, e em que casos a Huawei o indica. Se `/dev/kvm` existir, inicie uma micro-VM com o kernel e a imagem de exemplo da documentação.

T10.7 — Kernel e manutenção. Liste os kernels instalados e o padrão de boot. Explique, a partir da documentação, o que o recurso de atualização do kernel sem reinicialização (`nvwa`) faz e quais são suas limitações. Explique o que é o KAE e por que só faz sentido em processadores Kunpeng.

Comandos: `rpm -q kernel`, `grubby --info=ALL`, `grubby --default-kernel`.

T10.8 — Migração. Leia sobre o `x2openEuler`. Em até dez linhas, descreva o que a ferramenta avalia ao migrar um sistema CentOS para openEuler e quais são as etapas.

Perguntas para pensar

1. O A-Tune aplicou um perfil e o `rollback` desfez. Que risco existe em aplicar um perfil de ajuste em produção sem medir antes?
2. Contêiner, micro-VM e máquina virtual completa: compare isolamento, tempo de início e consumo de memória, e diga qual escolheria para o agente coletor.

Ponto de conferência 10

Tarefa	Verificação	Valor esperado
T10.2	`sysctl vm.swappiness net.core.somaxconn` após reiniciar	10; 4096
T10.3	`systemctl is-active atuned`	`active`
T10.3	`atune-adm list` após o `rollback`	Nenhum perfil com `Active` igual a `true`
T10.4	`isula ps -a` após o `rm`	Nenhum contêiner
T10.5	`curl -s -o /dev/null -w '%{http_code}' http://localhost:8081/`	200
T10.7	`grubby --default-kernel`	Caminho do kernel 6.6 em `/boot`

15. Desafio final (3 h)

A NuvemIF vai colocar o servidor em produção. Você entrega o servidor pronto, um script que prova que ele está pronto e um roteiro de operação. Não há lista de comandos: tudo o que é preciso foi visto nos dez módulos.

D1 — Serviço endurecido. Revise `coletor.service` com as proteções do `systemd`: sistema de arquivos somente leitura exceto `/var/log/nuvemif`, sem novos privilégios, sem acesso a `/home`, diretório temporário privado. Use `systemd-analyze security coletor` antes e depois e registre a nota.

D2 — Painel automático. Crie `/usr/local/bin/gera_painel.sh`, que lê o CSV e o log e grava `/srv/painel/index.html` com: nome da máquina, data e hora da geração, custo total do mês, os três projetos mais caros, o número de erros no log e o endereço que mais falhou na autenticação. Um temporizador o executa a cada 5 minutos. O `nginx` serve a página na porta 80, com SELinux em modo restritivo e contexto correto, e o firewall libera apenas `ssh` e `http`.

D3 — Backup com retenção. O backup diário do módulo 8 passa a guardar no máximo sete arquivos em `/mnt/backup`, apagando os mais antigos, e a incluir as unidades `systemd` e os scripts que você criou. Faça um teste de restauração em `/tmp/restaura` e compare com o original.

D4 — Acesso. Só `ana` e o seu usuário entram por SSH, apenas com chave. `ana` pode reiniciar o coletor e ler seus logs; ninguém além de `root` lê `segredos.env`. Toda mudança em `/opt/nuvemif/config` e em `/etc/systemd/system` fica registrada na auditoria.

D5 — Prova. Escreva `verificar.sh`, que executa pelo menos doze verificações e imprime `OK` ou `FALHA` para cada uma, terminando com código de saída 1 se alguma falhar. Entre elas: coletor ativo e habilitado; painel responde 200; página contém o custo total; temporizadores do painel e do backup ativos; backup de hoje existe; `getenforce` igual a `Enforcing`; firewall só com `ssh` e `http`; `sshd` sem senha e sem `root`; nenhum arquivo gravável por todos em `/opt/nuvemif`; `segredos.env` com `0600` ou `0640`; `vm.swappiness` igual a 10; regra de auditoria presente. Reinicie a máquina e execute o script: tudo precisa continuar `OK`.

D6 — Roteiro de operação. Em uma página, escreva para quem vai operar o servidor: como ver se está tudo bem, como reiniciar o coletor, onde estão os logs e os backups, como restaurar, e o que fazer se o painel responder 403.

Ponto de conferência final

Entrega	Verificação	Valor esperado
D1	`systemd-analyze security coletor`	Nota melhor que a inicial, com `NoNewPrivileges=yes` e `ProtectSystem=strict`
D2	`curl -s http://localhost/` contém	`65984.13`, `PRJ-004`, `594` e `198.51.100.23`
D2	`systemctl list-timers`	Painel a cada 5 min e backup diário
D3	Arquivos em `/mnt/backup` após dez execuções forçadas	7
D5	`./verificar.sh; echo $?` depois de reiniciar	Tudo `OK`; `0`

16. Avaliação e checklist de entrega

A nota de cada módulo segue os pesos da seção 2 e é distribuída por quatro critérios:

Critério	Peso	O que é observado
Correção	50%	Pontos de conferência atingidos, com comando e saída copiados do terminal
Raciocínio	25%	Perguntas para pensar respondidas com evidência do próprio sistema

Critério	Peso	O que é observado
Qualidade	15%	Scripts legíveis, com aspas, validação e códigos de saída; unidades `systemd` mínimas e corretas; nada feito como `root` sem necessidade
openEuler	10%	Recursos próprios usados onde o roteiro pede, com referência à documentação

Antes de entregar

- [ ] `relatorio.md` tem todas as tarefas, com código, comando e saída.
- [ ] A tabela dos pontos de conferência está preenchida com os meus valores.
- [ ] As perguntas para pensar dos dez módulos estão respondidas.
- [ ] Os scripts e as unidades `systemd` estão na pasta de entrega, com o caminho original anotado.
- [ ] `verificar.sh` termina com `0` depois de reiniciar a máquina.
- [ ] O instantâneo `lab-pronto` ainda existe, para o tutor reproduzir qualquer módulo.
- [ ] `segredos.env` não aparece em nenhuma saída copiada para o relatório.

17. Consulta rápida e documentação

Comandos por assunto

Assunto	Comandos
Sistema	`hostnamectl`, `uname`, `lscpu`, `free`, `lsblk`, `df`, `uptime`, `timedatectl`
Arquivos e texto	`find`, `grep`, `sed`, `awk`, `cut`, `sort`, `uniq`, `wc`, `tar`, `ln`, `diff`
Usuários	`useradd`, `usermod`, `passwd`, `chage`, `groupadd`, `id`, `getent`, `visudo`, `setfacl`, `getfacl`
Permissões	`chmod`, `chown`, `umask`, bits SUID, SGID e sticky
Processos e serviços	`ps`, `top`, `kill`, `nice`, `systemctl`, `journalctl`, `systemd-analyze`, `logrotate`
Pacotes	`dnf install`, `search`, `provides`, `history`, `updateinfo`, `rpm -q`, `-ql`, `-qf`, `-V`, `createrepo_c`

Assunto	Comandos
Rede	`ip`, `nmcli`, `ss`, `curl`, `ssh`, `scp`, `tcpdump`, `firewall-cmd`
Armazenamento	`parted`, `mkfs`, `mount`, `blkid`, `findmnt`, `pvcreate`, `vgcreate`, `lvcreate`, `lvextend`, `mkswap`
Automação	`bash`, `getopts`, `crontab`, `systemctl list-timers`, `set -euo pipefail`, `trap`
Segurança	`getenforce`, `semanage`, `restorecon`, `ausearch`, `auditctl`, `faillock`, `openEuler-security.service`
Desempenho e openEuler	`sysctl`, `vmstat`, `sar`, `atune-adm`, `isula`, `stratovirt`, `grubby`

O que é próprio do openEuler

Recurso	Para quê
Repositórios OS, everything, EPOL e update em `repo.openeuler.org`	Pacotes oficiais; EPOL reúne pacotes extras mantidos pela comunidade
A-Tune (`atuned`, `atune-engine`, `atune-adm`)	Identifica a carga de trabalho e aplica perfis de ajuste do sistema
iSulad (`isula`)	Motor de contêineres leve, compatível com imagens OCI e com a interface CRI do Kubernetes
StratoVirt	Virtualização leve: micro-VMs com início rápido e pouca memória, compatível com o QEMU
Ferramenta de endurecimento (`/etc/openEuler_security/`)	Aplica itens de segurança no primeiro boot e itens definidos pelo administrador
secGear	Desenvolvimento de aplicações em ambiente de execução confiável (TEE)
KAE	Aceleração de criptografia e compressão em processadores Kunpeng
Atualização do kernel sem reinicialização (nvwa)	Troca o kernel preservando os processos em execução

Recurso	Para quê
x2openEuler	Avalia e conduz a migração de CentOS para openEuler

Documentação

A documentação oficial está em docs.openeuler.org, organizada por versão: escolha a 24.03 LTS SP4. Os guias mais usados neste estudo são Installation, Administrator Guide (usuários, serviços, rede, LVM, dnf), Security Hardening, SELinux, A-Tune, Container (iSulad) e StratoVirt. As páginas `man` do próprio sistema são a referência para cada comando: `man systemd.service`, `man systemd.timer`, `man firewall-cmd`, `man semanage`, `man 5 sudoers`, `man 5 crontab`.

Para fundamentos de Linux em português, o Guia Foca Linux continua útil para a parte básica. Para `systemd`, a documentação do projeto é a fonte primária.